# Tree ADT

Generic methods:

- isEmpty(Tree t)
- create(Tree t)
- insertData(Tree t, object o)
- deleteData(Tree t, object o)
- destroy(Tree t)

Query methods:

- isInternal(Node p)
- isRight(Node p)
- isLeaf(Node p)
- isRoot(Node p)
- search(Tree t, object o)
- preOrder(Tree t)
- inOrder(Tree t)
- postOrder(Tree t)

Update methods:

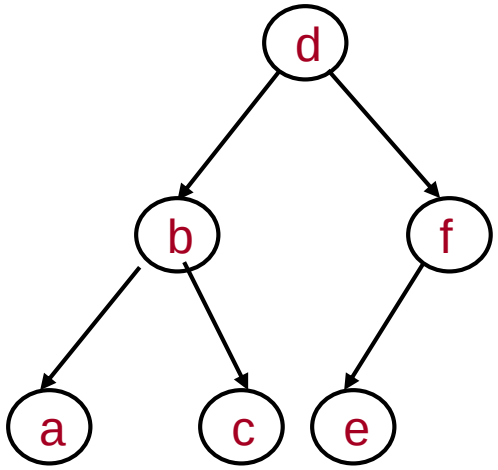
- swapElements(Node p, Node q)
- …

# Representation of a Binary Tree

- Array-based representation
- Reference-based representation

# Array-based representation

-1: empty tree




| nodeNum | item | leftChild | rightChild |
|---|---|---|---|
| 0 | d | 1 | 2 |
| 1 | b | 3 | 4 |
| 2 | f | 5 | -1 |
| 3 | a | -1 | -1 |
| 4 | c | -1 | -1 |
| 5 | e | -1 | -1 |
| 6 | | | |
| 7 | | | |
| 8 | | | |
| 9 | | | |
| ... | ..... | ..... | .... |

root

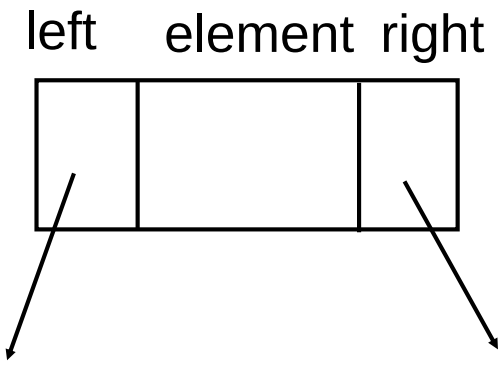
0

free

6

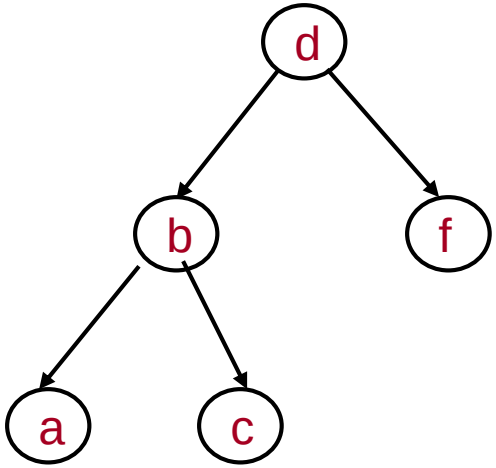
# Reference Based Representation

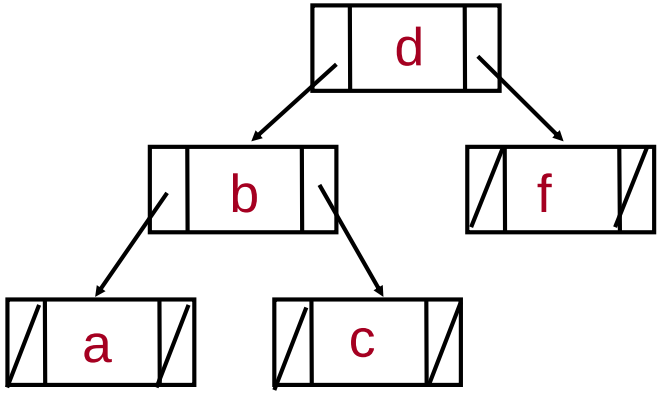
NULL: empty tree




You can code this with a class of three fields:

```
Object element;
TreeNode left;
TreeNode right;
```



# Binary Tree ADT

- The BinaryTree ADT extends the Tree ADT, i.e., it inherits all the methods of the Tree ADT

- Additional methods:
  - leftChild(p)
  - rightChild(p)
  - sibling(p)

# Binary Tree Traversal

The traversal must start at the root because one is typically given a reference to the root. For each vertex, there are two choices.

- Visit the vertex first.
- Visit the subtrees first.

# Tree Traversal – Binary Tree

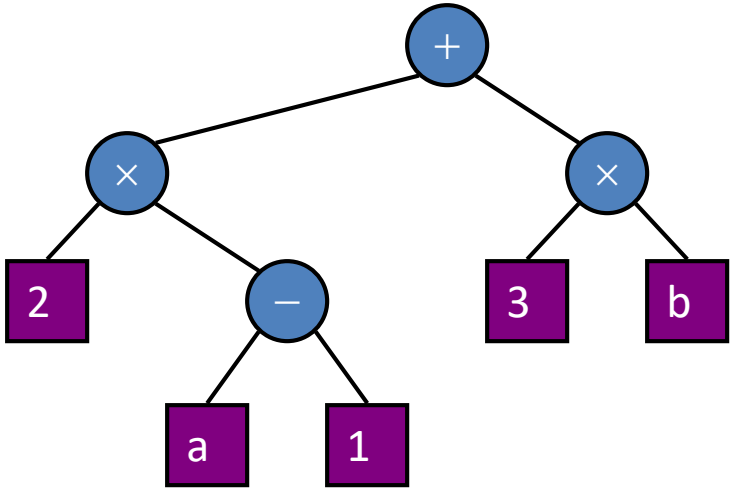
- The usual notation for writing arithmetic expressions is called **infix** notation, in which the operator is written between the operands.

- For example, in the expression a + b, the operator + is between the operands a and b. In infix notation, the operators have precedence. For example, expressions are evaluated from left to right, and multiplication and division have higher precedence than addition and subtraction.

# Example

- Binary tree associated with an arithmetic expression
  - internal nodes: operators
  - external nodes: operands

- Example: arithmetic expression tree for the expression $(2 \times (a - 1) + (3 \times b))$



# Tree Traversal – Binary Tree

If operators were written before the operands (**prefix**, for example, +xy) or after the operands (suffix, **postfix**, for example, xy+), the parentheses could be omitted.

For example, the expression (x + y) * z in a postfix expression is zxy+*

# Binary Tree Traversal

- inorder traversal.
- preorder traversal.
- postorder traversal.

# Binary Tree Traversal

**Inorder Traversal**: In this case, the binary tree is traversed as follows:

1. Traverse the left subtree (L).
2. Visit the vertex (D).
3. Traverse the right subtree (R).

# Algorithm : Inorder traversal

INPUT: root - a reference to the root of the binary tree

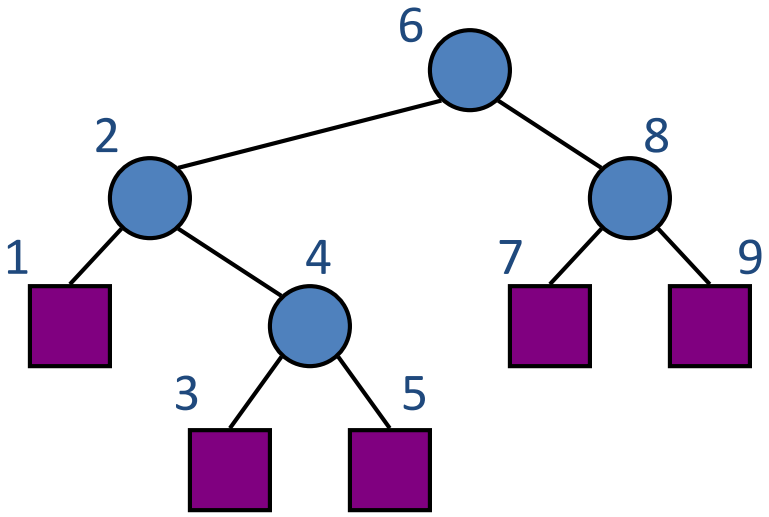
OUTPUT: vertices of the binary tree inorder sequence

```
Inoder(root)
If root is not NULL then
        Inorder(left(root));
        print root;
        Inorder(right(root));
```

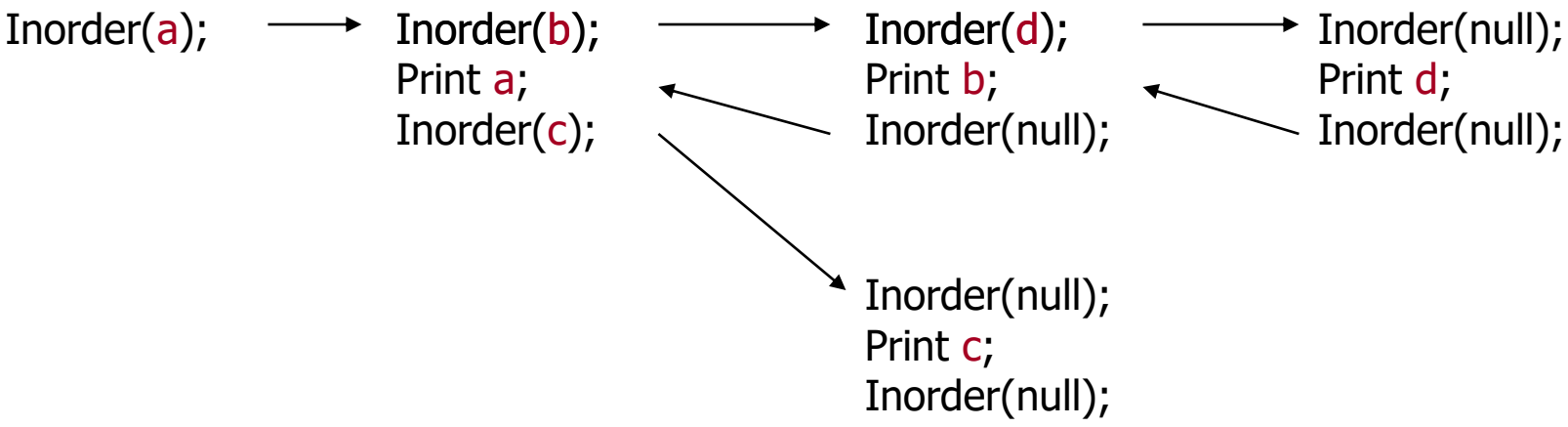
# Inorder Traversal

- In an inorder traversal **a node is visited** after its left subtree and before its right subtree

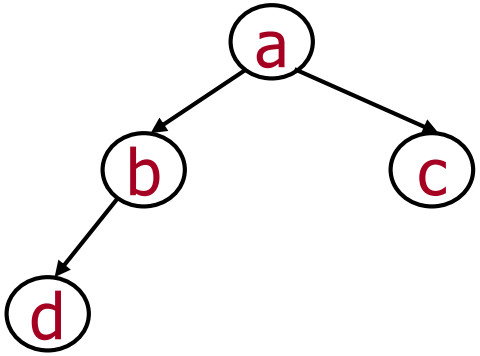
```
Algorithm inOrder(v)
    if isInternal (v)
        inOrder (leftChild (v))
    visit(v)
    if isInternal (v)
        inOrder (rightChild (v))
```



# Inorder example (LDR)

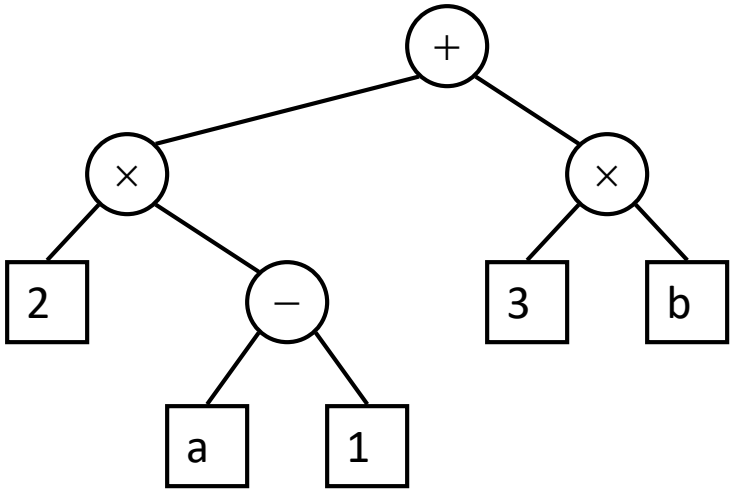



d b a c



# Print Arithmetic Expressions

```
Algorithm printExpression(v)
    if isInternal (v)
        print("(")
        inOrder (leftChild (v))
    print(v.element ())
    if isInternal (v)
        inOrder (rightChild (v))
        print (")")
```




((2 × (a − 1)) + (3 × b))

# Binary Tree Traversal

**Preorder Traversal:** In a preorder traversal, the binary tree is traversed as follows:

1. Visit the vertex (D).

2. Traverse the left subtree (L).

3. Traverse the right subtree (R).

# Algorithm : Preorder traversal

INPUT: root - a reference to the root of the binary tree

OUTPUT: vertices of the binary tree preorder sequence

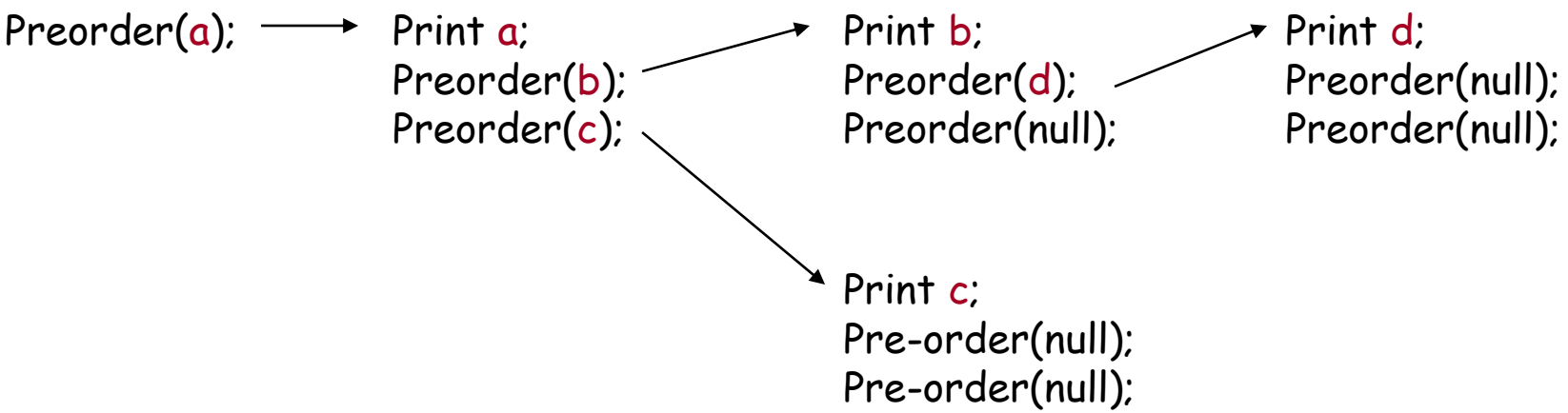
```
Preoder(root)
If root is not NULL then
        print root;
        Preorder(left(root));
        Preorder(right(root));
```

# Preorder Traversal

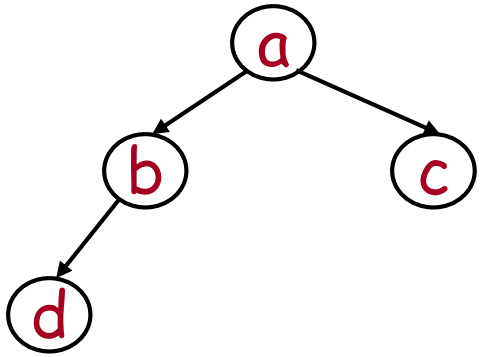
In a preorder traversal, a node is visited before its descendants

```
Algorithm preOrder(v)
    visit(v)
    for each child w of v
        preorder (w)
```

# Preorder example (DLR)




a b d c



# Binary Tree Traversal

**Postorder Traversal:** In a postorder traversal, the binary tree is traversed as follows:

1. Traverse the left subtree (L).
2. Traverse the right subtree (R).
3. Visit the vertex (D).

# Algorithm : Postorder traversal

INPUT: root - a reference to the root of the binary tree

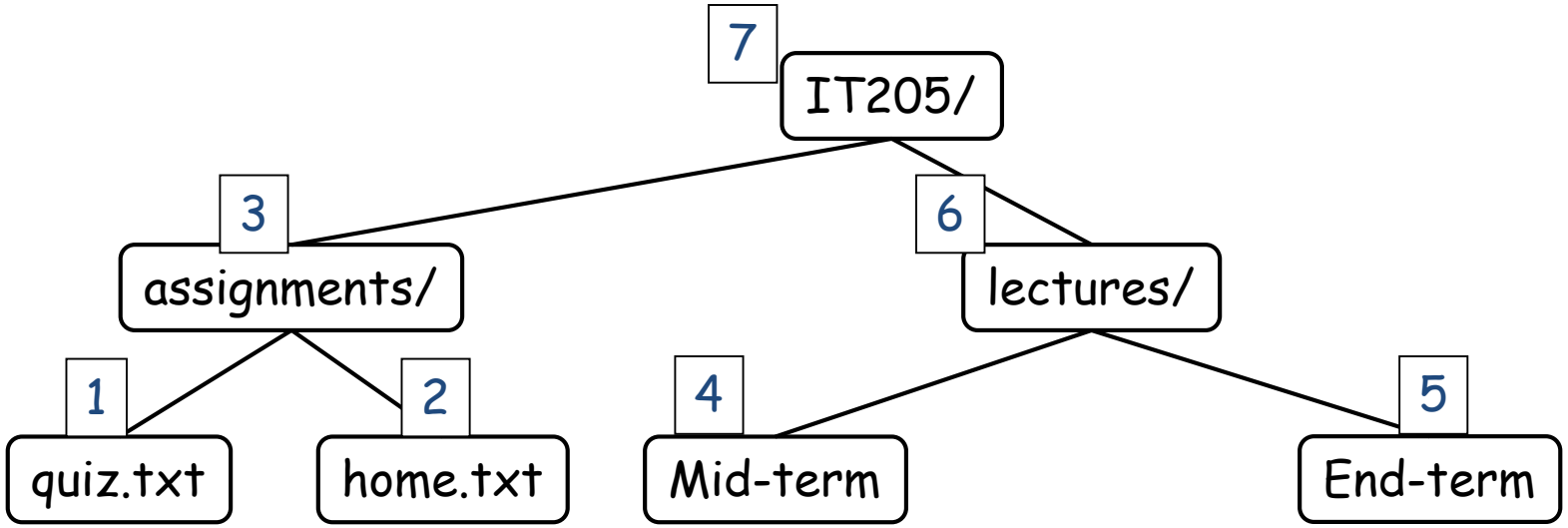
OUTPUT: vertices of the binary tree postorder sequence

```
Postoder(root)
If root is not NULL then
        Postorder(left(root));
        Postorder(right(root));
        print root;
```

# Postorder Traversal

- In a postorder traversal, a node is visited after its descendants
- Application: memory used by files in a directory and its subdirectories

```
Algorithm postOrder(v)
    for each child w of v
        postOrder (w)
    visit(v)
```



# Task

- By inorder, (infix)
- By preorder, (prefix)
- By postorder, (postfix)

